

第4章 函数与方法

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握函数定义语法:fn 关键字、参数、返回类型、表达式与语句的区分
- 深入理解 参数传递的所有权模型:传值拷贝 vs 所有权移动 vs 借用——C 的"传值 + 指针模拟引用"如何被 Rust 的显式语义取代
- 掌握多返回值(元组)与解构接收;了解返回引用的限制(第5章展开生命周期)
- 掌握方法:impl 块、self/&self/&mut self 三态、关联函数(构造函数惯例)
- 掌握闭包语法与 捕获三态 Fn / FnMut / FnOnce,理解 move 关键字,体会闭包如何超越 C 的函数指针
- 掌握高阶函数:map/filter/fold、函数作为参数与返回值
- 理解 Rust 没有重载与默认参数,掌握生产替代方案(泛型、配置结构体、不同函数名)
- 了解递归与栈溢出、main 的形态(env::args 替代 argc/argv)、Result 错误返回约定

💡 从 C 到 Rust:本章主题如何迁移

C 的函数只有一种"传参哲学":值传递。想修改外部,就传指针;想传大结构体,也传指针。于是每个函数签名都要靠程序员心算"这个指针会不会被改、生命周期到哪"——信息全在注释和约定里,编译器一概不管。Rust 把参数传递分成三种显式语义:按值移动(move)、不可变借用(&T)、可变借用(&mut T),写进签名,由编译器强制执行。C 程序员最需要改的习惯是:"传指针"在 Rust 里要明确回答——你要借来看,还是要借去改,还是要整个拿走?

C 的函数指针是二等公民:能存、能传,但捕获不了上下文(回调需要额外的 void* 参数手工传递状态)。Rust 的闭包把"函数 + 捕获的环境"打包成一个整体,而且捕获方式受控(三态 Fn/FnMut/FnOnce)。你在 C 里见过的 `qsort(data, n, sizeof(int), cmp)` 式回调,在 Rust 里变成 `data.sort_by(|a, b| a.cmp(b))` ——更短,且不依赖任何全局状态。

Rust 没有重载、没有默认参数、没有可变参数(printf 式)。初看像倒退,实际是把"同名不同义"的暧昧从语言里赶走:重载造成的隐式转换与歧义(C++ 里 `f(1)` 该调哪个 `f`?) 在 Rust 中不存在,替代品是泛型(第10章)、配置结构体和 Builder。函数签名于是成为精确契约:看到 `fn f(x: &str) -> Result<i32, Err>`,你就知道它借用一个字符串、可能失败——这比 C 里任何注释都可靠。

最后是"入口与出口":Rust 的 main 没有 argc/argv(用 `env::args()` 获取),函数没有 `errno`(错误通过返回值 `Result` 显式传播,第9章详解)。整章的主线可以概括为一句话:函数的一切约定,都从"注释里的口头协议"变成"签名里的类型契约"。

📦 前置知识自查

- C 的函数定义、值传递、指针参数、返回语句
- C 的函数指针:typedef、回调(如 `qsort` 的 `cmp`)
- C 的 `argc/argv`、`errno` 错误返回约定
- C 的 `struct` 与"传结构体指针"惯用法
- 第1-3章:所有权伏笔、Option、match、迭代器基本遍历

本章学习路线

1. **第一阶段(约1小时):**§4.1–§4.3,吃透参数传递三态——本章最核心的概念
2. **第二阶段(约1.5小时):**§4.4–§4.6,方法与闭包;闭包三态多写几遍直到形成直觉
3. **第三阶段(约1.5小时):**§4.7–§4.8 与章末实验,把高阶函数用进真实管线
4. **验收标准:**能说出"传 &T、传 &mut T、传 T"各适用什么场景;能写出 Fn/FnMut/FnOnce 三个闭包示例

本章知识导图

- §4.1 函数定义与调用:fn、参数、返回、表达式风格
- §4.2 参数传递:所有权移动 vs 借用(全对照表)
- §4.3 返回值:多返回值、引用返回与生命周期预告
- §4.4 方法:impl、self 三态、关联函数
- §4.5 闭包:语法、捕获三态 Fn/FnMut/FnOnce、move
- §4.6 高阶函数:函数指针、map/filter/fold、回调
- §4.7 无重载无默认参数:替代方案;递归与栈溢出
- §4.8 入口与错误返回:main、env::args、Result 约定
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§4.1 函数定义与调用

4.1.1 基本形态

函数以 `fn` 开头,参数必须注解类型,返回类型用 `->` 声明(没有返回值时省略)。函数体是表达式风格:最后一个不带分号的表达式就是返回值,等价于 C 的 `return`:

```
// C: int add(int a, int b) { return a + b; }
fn add(a: i32, b: i32) -> i32 {
    a + b                // 无分号:这是函数的返回值
}

// 无返回值:C 的 void;实际返回单元类型 ()
fn greet(name: &str) {
    println!("你好,{name}");
    // 没有 return 语句
}

// 显式 return 也合法:提前返回时用
fn abs(n: i32) -> i32 {
    if n < 0 {
        return -n;        // 提前返回
    }
    n
}

fn main() {
    println!("{}", add(3, 4));    // 7
    greet("张三");
    println!("{}", abs(-5));      // 5
}
```

4.1.2 语句与表达式:函数体是"表达式串"

每个函数体可以看作一列"表达式 + 分号":带分号的是语句(执行副作用,值被丢弃),最后一个不带分号的表达式是函数的值。C 程序员最常见的编译错误 "expected `)` , found `i32`" 或相反,根源都是"最后一行该不该加分号"——检查顺序:函数声明了 `-> i32`,最后一行就必须是 `i32` 表达式,不能加分号。

示例 4-1 三种返回值写法对照

```
fn f1() -> i32 {
    return 42;                // C 风格:显式 return
}

fn f2() -> i32 {
    42                        // Rust 风格:尾表达式
}

fn f3() -> i32 {
    let x = 40;
    x + 2                    // 尾表达式可以是任意表达式
}

fn main() {
    println!("{}", f1(), f2(), f3());
}
```

§4.2 参数传递:移动与借用(本章核心)

4.2.1 C 的"值 + 指针"二分 vs Rust 的"移动 + 借用"三分

C 的规则很简单:一切按值拷贝;想影响外部就传指针。但"传指针"到底是只读还是可写,靠的是注释和程序员自觉。Rust 把参数传递显式分成三种,写进类型签名:

C 的写法	Rust 的写法	语义
<code>void f(int x)</code>	<code>fn f(x: i32)</code>	传值:内部修改不影响外部(C 同理)
<code>void f(const int *p)</code>	<code>fn f(x: &i32)</code>	不可变借用:只看不改(C 靠 <code>const</code> 约定)
<code>void f(int *p)</code>	<code>fn f(x: &mut i32)</code>	可变借用:可修改外部(编译器强制独占)
<code>void f(Item *p)</code> (大结构体)	<code>fn f(x: &Item)</code>	只读借用,不拷贝(替代 C 的传指针省拷贝)
(无对应)	<code>fn f(x: Item)</code>	所有权移动:数据被搬进函数,调用后原变量失效

4.2.2 移动语义:值被"搬走"而不是"拷贝"

第5章会系统讲所有权,这里先看行为:对 `String`、`Vec` 等"拥有堆数据"的类型,按值传参意味着所有权从调用方转移到函数内部;调用后原变量不再可用(编译器报 `moved` 错误)。对 `i32` 等 `Copy` 类型,传参就是拷贝,原变量照常可用:

```
fn take(s: String) {
    println!("收到:{s}");
} // 离开作用域,String 被自动释放(第5章的 drop)

fn main() {
    let name = String::from("张三");
    take(name);                // 所有权被搬进 take
    // println!("{name}");     // 编译错误:value borrowed after move
    let n = 5;
    add_and_print(n);          // i32 是 Copy:只是拷贝
    println!("{n}");           // 5,照常可用
}

fn add_and_print(x: i32) {
    println!("{}", x + 1);
}
```

4.2.3 借用参数:&T 与 &mut T

当函数只需要"看"数据时,传 `&T` (不可变借用)——不拷贝大对象,也不搬走所有权;需要"改"外部数据时,传 `&mut T`。C 里 `f(&x)` 之后 `x` 的命运全靠约定,Rust 里签名直接写死:

```
// 只读借用:可以传数组、Vec、String 的任何"视图"
fn total(v: &[i32]) -> i32 {
    let mut s = 0;
    for &x in v {
        s += x;
    }
    s
}

// 可变借用:直接修改外部数据,不需要 C 的解引用链
fn bump(x: &mut i32, by: i32) {
    *x += by;           // *x 解引用,与 C 的 *p 相同
}

fn main() {
    let arr = [1, 2, 3, 4];
    println!("{}", total(&arr));           // 10:传 &arr 借出

    let mut score = 80;
    bump(&mut score, 5);                   // 修改外部变量
    println!("{}", score);                 // 85
}
```

⚠ 易错点 1:传参时漏写 &,或把 &mut 当 & 用

三种报错天天见:① `fn f(x: &i32)` 却写 `f(x)` ——编译器提示 "expected &i32, found i32",补上 `&x`;② 函数签名是 `&mut i32`,调用处必须 `&mut x`,且调用前 `x` 必须声明为 `mut`;③ 把可变借用的值再传给只读函数没问题(可以同时借用),但反过来——函数内先拿了 `&mut` 又想读——要按借用规则排队(第5章详讲)。先记住一句话:函数要"看"就传 `&`,要"改"就传 `&mut`,要"拿走"就传值。

§4.3 返回值

4.3.1 多返回值:C 的"输出参数"被元组取代

C 里函数要返回多个结果,要么用输出参数 `int div(int a, int b, int *q, int *r)`,要么定义一个临时结构体。Rust 用元组直接返回,调用处解构:

```
// 返回 (商, 余数):元组类型写进签名
fn div_mod(a: i32, b: i32) -> (i32, i32) {
    (a / b, a % b)
}

fn main() {
    let (q, r) = div_mod(17, 5);           // 解构接收:q=3, r=2
    println!("商 {q},余数 {r}");

    // 也可以整体存下再取下标
    let result = div_mod(20, 3);
    println!("{}", ... {}, result.0, result.1);
}
```

4.3.2 返回引用:生命周期问题初现

函数可以返回借用,但编译器要能证明"返回的引用不会悬垂"。这个证明就是生命周期标注(第5章详讲),这里先看两个形态:返回参数里的借用(合法),返回局部变量的借用(编译错误):

```
// 合法:返回的引用来自参数(编译器能证明它活得比函数长)
fn first_word(text: &str) -> &str {
    match text.find(' ') {
        Some(pos) => &text[..pos],
        None => text,
    }
}

fn main() {
    let s = String::from("hello world");
    let w = first_word(&s);
    println!("第一个单词:{w}");

    // 编译错误:返回局部变量的引用(悬垂引用!)
    // fn bad() -> &String {
    //     let s = String::from("x");
    //     &s    // s 在函数结束时被释放,引用悬空
    // }
}
```

§4.4 方法:impl 块

4.4.1 方法与 self 三态

C 把"对结构体的操作"写成 `rect_area(&r)` 之类的自由函数;Rust 允许把函数挂在类型上成为**方法**,第一个参数是 `self`。self 有三种形态: `self`(拿走所有权)、`&self`(只读)、`&mut self`(可变) ——与参数传递三态完全一致:

```

struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    // &self:只读方法(C 的 const 指针版本)
    fn area(&self) -> f64 {
        self.width * self.height
    }

    // &mut self:修改自身
    fn scale(&mut self, factor: f64) {
        self.width *= factor;
        self.height *= factor;
    }

    // self:消费自身(拿走所有权)
    fn into_square(self) -> Rectangle {
        Rectangle { width: self.width, height: self.width }
    }
}

fn main() {
    let mut r = Rectangle { width: 3.0, height: 4.0 };
    println!("面积:{}", r.area());           // 12(调用后 r 仍可用)
    r.scale(2.0);                             // 修改 r
    println!("放大后:{}", r.area());          // 48
    let sq = r.into_square();                  // r 被消费
    // println!("{}", r.area());              // 编译错误:r 已 move
    println!("正方形:{}", sq.area());         // 64
}

```

4.4.2 关联函数:没有 self 的"静态方法"

不接收 self 的函数挂在 impl 块里,称为**关联函数**,用 `::` 调用;惯例上以 `new` 命名的关联函数充当构造函数——这替代了 C 里"手写 init 函数 + 调用者负责初始化"的脆弱模式:

```

struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // 关联函数(构造函数惯例):Point::new(...)
    fn new(x: f64, y: f64) -> Point {
        Point { x, y }
    }

    fn origin() -> Point {
        Point { x: 0.0, y: 0.0 }
    }

    fn dist(&self, other: &Point) -> f64 {
        let dx = self.x - other.x;
        let dy = self.y - other.y;
        (dx * dx + dy * dy).sqrt()
    }
}

fn main() {
    let a = Point::new(1.0, 1.0);           // 关联函数:类型名::函数名
    let b = Point::origin();
    println!("距离:{}", a.dist(&b));
}

```



工程建议:方法 vs 自由函数的选择

- 操作"明显属于"这个类型(读字段、改字段):方法。C 的 `strlen(s)` 在 Rust 里是 `s.len()`
- 两个类型协同的操作(如 `point.dist(&other)`):方法更自然
- 与类型关系不强的纯计算:自由函数,避免"万物皆方法"的 Java 式强迫症
- 构造函数一律叫 `new`,是社区惯例;需要"解析/尝试构造"时用 `from_str` / `try_from` (第1章见过)

§4.5 闭包:捕获环境的函数

4.5.1 语法:参数与体

```

fn main() {
    // C 函数指针:int (*f)(int);Rust 闭包:
    let double = |x| x * 2;           // 参数类型可推导(常用)
    let greet = |name: &str| { println!("你好,{name}") }; // 多语句用 {} 包裹
    println!("{}", double(4));       // 8
    greet("李四");
}

```

闭包的类型无法显式书写(编译器自动推断),这反而是优点:你只需要写参数与体。闭包与 C 函数指针的本质区别:闭包能捕获定义处的变量(上下文),函数指针不能。C 要模拟捕获,只能靠额外的 `void*` 参数。

4.5.2 捕获三态:Fn / FnMut / FnOnce(必会)

闭包按"如何使用捕获的变量"分成三类,对应三个 trait(第6章详解 trait;这里记住行为和名字):

trait	捕获方式	能调用几次	例子
Fn	只读借用(&T)	任意多次	x x + base(只读 base)
FnMut	可变借用(&mut T)	任意多次	x { total += x; }(改 total)
FnOnce	拿走所有权(move)	一次	move drop(owner)

```
fn main() {
    // Fn:只"看"不"动"
    let base = 10;
    let add = |x| x + base;           // 捕获 &base
    println!("{}", add(1), add(2)); // 可反复调用

    // FnMut:闭包内部修改捕获的变量(计数器模式)
    let mut total = 0;
    let mut accumulate = |x: i32| {
        total += x;                 // 捕获 &mut total
    };
    accumulate(5);
    accumulate(7);
    println!("累计:{total}");       // 12(借用已结束)

    // FnOnce:把值搬进闭包,之后外部不可再用
    let owner = String::from("机密数据");
    let consume = move || println!("{}", owner); // move:彻底搬走
    consume();
    // consume();           // 编译错误:FnOnce 只能调用一次
    // println!("{}", owner); // 编译错误:已被搬走

    // 判断口诀:用到值就得 move(FnOnce);改值就是 FnMut;只看就是 Fn
}
```

4.5.3 move 关键字:主动搬入

闭包默认"按需借用"捕获;用 `move` 强制把所有捕获值搬进闭包。最典型的场景是 把数据交给新线程(第9章)——线程可能活得比函数长,借用必然悬垂,必须 `move`:

```
fn main() {
    let data = vec![1, 2, 3];
    // 不 move:闭包借用了 data,离开作用域后 data 被释放,闭包悬垂
    // let bad = || println!("{}", data:?);

    // move:data 的所有权进入闭包,闭包生命周期自持
    let owned = move || println!("{}", data:?);
    owned();
    // [1, 2, 3]
    // println!("{}", data:?); // 编译错误:已 move 进闭包
}
```

⚠ 易错点 2: 闭包调用次数与捕获方式的错配

经典报错:① "cannot move out of `owner`, a captured variable in an `Fn` closure"——闭包没写 move 却要消费值,给闭包加 move 即可;② "cannot borrow `total` as mutable, as it is not declared as mutable"——闭包要改捕获变量,闭包本身也要声明 mut;③ 把一个 FnOnce 闭包存进 Vec 想循环调用——编译器告诉你只能调一次,要么换 FnMut,要么重新设计。三态的区分不是考试题,是编译器每天都在执行的检查:它知道你的闭包会怎么用捕获的变量。

§4.6 高阶函数:函数作为参数与返回值

4.6.1 函数指针类型 fn 与回调

函数名本身就是值,类型写作 `fn(i32) -> i32` (与 C 的函数指针——对应,但写法不带星号)。接受函数参数的函数叫高阶函数:

```
// 接受"处理函数"作参数:C 的 void apply(int *a, int n, int (*f)(int))
fn apply_each(data: &[i32], f: fn(i32) -> i32) -> Vec<i32> {
    let mut out = Vec::new();
    for &x in data {
        out.push(f(x));
    }
    out
}

fn triple(x: i32) -> i32 {
    x * 3
}

fn main() {
    let nums = [1, 2, 3, 4];
    let tripled = apply_each(&nums, triple); // 传函数名
    println!("{tripled:?}");                // [3, 6, 9, 12]

    // 函数指针也是值:可以存、可以传
    let f: fn(i32) -> i32 = triple;
    println!("{}", f(5));                    // 15
}
```

4.6.2 迭代器方法链:map / filter / fold

标准库把"遍历 + 变换 + 归约"做成了迭代器方法,配合闭包使用——这是 C 里必须手写循环三件套(map 之于 for 循环,filter 之于 if 判断,fold 之于累加):

```
fn main() {
    let data = [3, 5, 8, 12, 15, 20];

    // map:每个元素变换(等价 C 的"循环里 push 新值")
    let doubled: Vec<i32> = data.iter().map(|x| x * 2).collect();
    println!("{doubled:?}");

    // filter:按条件筛选(等价 C 的"循环里 if 判断再 push")
    let evens: Vec<i32> = data.iter().filter(|&&x| x % 2 == 0).collect();
    println!("{evens:?}");

    // fold:归约成单个值(等价 C 的累加器)
    let sum: i32 = data.iter().fold(0, |acc, &x| acc + x);
    println!("{sum}");

    // 链式组合:筛选 → 变换 → 求和,一次遍历
    let result: i32 = data.iter()
        .filter(|&&x| x > 5)
        .map(|x| x * x)
        .fold(0, |acc, x| acc + x);
    println!("{result}");    // 8^2+12^2+15^2+20^2 = 713
}
```

注意 `filter` 的闭包参数是 `&&i32` (迭代器产生引用,filter 又传引用),写法 `|&&x|` 直接解构——这是 Rust 迭代器中最常见的一个"仪式感",见多了自然记住。

§4.7 没有重载与默认参数:替代方案

4.7.1 重载的替代

C 没有重载(靠不同函数名),C++ 有静态重载。Rust 没有重载,但有更系统的替代:① 泛型 (第10章)——`fn max<T: PartialOrd>(a: T, b: T) -> T` 一个函数服务所有类型;② 枚举参数——把"同操作不同细节"收进一个枚举;③ 不同函数名——`from_bytes`、`from_str` 明确区分。结论:重载带来的"同名歧义"被消灭,代价是函数名略长。

4.7.2 默认参数的替代:配置结构体

```
// C 的典型场景:draw(color, width, style) 想省略参数—用宏或重载;
// Rust 的做法:把可选项收进配置结构体,配合 Default 与更新语法
struct DrawOpts {
    color: u32,
    width: u32,
    style: &'static str,
}

impl DrawOpts {
    fn default() -> Self {
        DrawOpts { color: 0x000000, width: 1, style: "solid" }
    }
}

fn draw(opts: &DrawOpts) {
    println!("画 {}, 颜色 0x{:06X},{:02}px", opts.style, opts.color, opts.width);
}

fn main() {
    let defaults = DrawOpts::default();          // 全默认
    draw(&defaults);

    // 只改一个参数:..defaults 更新语法(第14章详解)
    let red = DrawOpts { color: 0xFF0000, ..defaults };
    draw(&red);
}
```

4.7.3 递归与栈溢出

```
fn factorial(n: u64) -> u64 {
    if n <= 1 {
        return 1;
    }
    n * factorial(n - 1)
}

fn main() {
    println!("{}", factorial(5));    // 120

    // 深递归会栈溢出:Rust 与 C 一样使用系统栈(默认 8MB),
    // 且不保证尾递归优化(TCO)—深递归请改写为循环
    let mut result: u64 = 1;
    let mut i = 20;
    while i > 1 {
        result *= i;
        i -= 1;
    }
    println!("{}", result);
}
```

§4.8 入口与错误返回约定

4.8.1 main 的形态与命令行参数

```
use std::env;

// C 的 int main(int argc, char *argv[]) → Rust 的 fn main()
fn main() {
    // argc/argv 由 env::args() 提供:返回迭代器
    let args: Vec<String> = env::args().collect();
    println!("程序名:{}", args[0]);
    if args.len() > 1 {
        for (i, a) in args.iter().enumerate().skip(1) {
            println!("参数 {i}:{a}");
        }
    } else {
        println!("没有参数;用法:程序名 <参数1> ...");
    }
}
```

4.8.2 错误返回约定:Result 取代 errno

C 的错误返回约定靠 `errno` + 返回码,调用链上每层都要"检查、传播、清理",漏一层就静默失败。Rust 的约定是 **函数签名里直接声明可能失败的返回值类型 `Result<T, E>`** (第9章系统讲):

```
// C: int divide(int a, int b, int *out); 返回 errno,成功与否查全局
// Rust:失败与否写在类型里,调用方必须处理
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        return Err("除数为零".to_string());
    }
    Ok(a / b)
}

fn main() {
    match divide(10.0, 2.0) {
        Ok(v) => println!("结果:{v}"),
        Err(e) => println!("错误:{e}"),
    }
    match divide(1.0, 0.0) {
        Ok(v) => println!("结果:{v}"),
        Err(e) => println!("错误:{e}"),
    }
}
```

★ 重点:本章贯穿线索——签名即契约

回顾全章:参数是 `&T` 还是 `&mut T` 还是 `T`,方法取 `&self` 还是 `self`,闭包是 `Fn` 还是 `FnOnce`,返回值是 `i32` 还是 `Result<i32, E>` —— 每个决定都写在签名里,编译器逐条执行。C 程序里那些"看注释才知道的约定",在 Rust 里都是类型。写函数时先想清楚签名,再写体;这是 Rust 生产开发的头号习惯。

⚠ 易错点 3:需要"函数做参数"时,优先闭包而不是 fn 指针

fn 指针类型 `fn(i32) -> i32` 不能捕获环境(它只是裸函数地址);闭包能捕获。生产里"回调参数"的签名优先写成泛型 `F: Fn(...)` 或 `&impl Fn(...)`,除非明确不需要捕获。对照 C:函数指针是唯一选项,所以 C 的回调都要靠"上下文指针"参数传状态;Rust 的闭包把"上下文"打包进闭包自身,少一个容易接错的 `void*`。

本章速查表

主题	结论 / 写法
函数定义	<code>fn name(a: i32, b: i32) -> i32 { ... }</code> ;无返回值省略 <code>-></code>
返回值	最后一个不带分号的表达式; <code>return</code> 用于提前返回
传值	<code>fn f(x: i32):Copy</code> 类型拷贝; <code>String/Vec</code> 等是所有权移动
不可变借用	<code>fn f(x: &T)</code> :只看不改;不拷贝大对象
可变借用	<code>fn f(x: &mut T)</code> :可修改外部;调用处写 <code>&mut x</code>
多返回值	<code>fn f() -> (i32, i32)</code> ;调用处 <code>let (a, b) = f();</code>
返回引用	只能返回参数/外部借来的引用;返回局部引用是编译错误
方法	<code>impl T { fn m(&self) {} }</code> ;三态 <code>self / &self / &mut self</code>
关联函数	<code>impl T { fn new() -> T {...} }</code> ;调用 <code>T::new()</code>
闭包	<code> x x + 1</code> ;可捕获上下文;参数类型多可推导
Fn	只读捕获,可多次调用
FnMut	可变捕获(闭包要声明 <code>mut</code>),可多次调用
FnOnce	<code>move</code> 捕获,只能调用一次
高阶函数	<code>fn</code> 类型可作参数; <code>map/filter/fold</code> 方法链
重载/默认参数	无!替代:泛型、配置结构体 + <code>Default</code> 、不同函数名
递归	支持,但无 <code>TCO</code> 保证;深递归用循环改写
main / 参数	<code>fn main();env::args()</code> 替代 <code>argc/argv</code>
错误返回	<code>Result<T, E></code> 写进签名; <code>match</code> 或 <code>?</code> 处理(第9章)

最小必会清单

- 能写出三种参数形态(传值/&/&mut)并说出各自语义与适用场景
- 能解释"String 传参后原变量失效"并说出原因(所有权移动)
- 能用元组返回多个值并在调用处解构
- 能写一个 `struct + impl`,包含 `&self` 方法、`&mut self` 方法、关联函数 `new`
- 能写出 `Fn/FnMut/FnOnce` 各一个闭包示例,并解释捕获方式差异
- 能说出 `move` 关键字的作用与"交给线程"场景
- 能用 `map/filter/fold` 组合完成一次数据管线处理

- 能说出 Rust 无重载/无默认参数时三种替代方案
- 能用 `env::args()` 读取命令行参数
- 能写出返回 `Result` 的函数并用 `match` 处理

自测题

Q 1. `fn f(s: String) { ... }` 与 `fn g(s: &String) { ... }` 调用后,原变量还能用吗?为什么?

✓ 参考答案

`f` 按值传参:`String` 的所有权被搬进函数,调用后原变量失效(编译错误 `moved`)。 `g` 只借用,`&String` 不转移所有权,原变量照常使用。选择依据:函数是否需要拿走数据;只需查看时永远优先 `&`。

Q 2. C 的 `void swap(int *a, int *b)` 用 Rust 怎么写?与 C 相比少做了什么?

✓ 参考答案

`fn swap(a: &mut i32, b: &mut i32) { std::mem::swap(a, b); }`。与 C 相比:不需要手动解引用赋值(用标准库 `swap`),调用处必须写 `&mut x`——编译器保证两个可变借用不冲突,杜绝 C 里"同一个变量传两个指针"的别名问题。

Q 3. 下面闭包属于 `Fn`、`FnMut` 还是 `FnOnce`? `let c1 = |x| x * 2;`、`let mut n = 0; let c2 = |x| { n += x; };`、`let s = String::from("a"); let c3 = move || drop(s);`

✓ 参考答案

`c1` 不捕获任何变量(或只读),是 `Fn`; `c2` 修改捕获的 `n`,是 `FnMut`; `c3` 用 `move` 搬走 `s` 并消费,是 `FnOnce`。判断顺序:会不会消费值(会→`FnOnce`)、会不会修改(会→`FnMut`)、否则 `Fn`。

Q 4. 用 `map/filter/fold` 把 `[1, 2, 3, 4, 5, 6]` 中偶数的平方求和(期望 `4+16+36=56`)。

✓ 参考答案

`let sum: i32 = [1,2,3,4,5,6].iter().filter(|&&x| x % 2 == 0).map(|x| x * x).fold(0, |acc, x| acc + x);` 结果 56。注意 `filter` 的闭包形参是 `&&i32`。

Q 5. Rust 没有默认参数,`draw(color, width, style)` 想省略参数时该怎么办?

✓ 参考答案

把可选项收进配置结构体 `DrawOpts`,实现 `Default`,调用处 `DrawOpts { color: 0xFF0000, ..defaults }` 只覆盖需要的字段。相比 C++ 默认参数,类型契约更清晰:配置本身是类型,IDE 与编译器都能参与检查。

Q 6. 为什么 Rust 的闭包能超越 C 的函数指针?C 里如何模拟"捕获"?

✓ 参考答案

函数指针只是地址,不能携带上下文;C 用"回调 + `void*` 参数"模拟,类型安全全靠自觉。Rust 闭包把"代码 + 捕获的环境(按 `Fn/FnMut/FnOnce` 受控)"打包,类型系统管理捕获方式,编译器检查借用与所有权。生产中最直接的好

处:sort_by、线程 spawn、迭代器方法链都无需手工传递上下文。

章末实验题:可配置的数组统计管线

实验 4:可配置的数组统计管线

实验目标:实现一个"配置驱动"的数据统计管线:用配置结构体控制统计规则,用函数 + 闭包 + 高阶函数完成多路统计,覆盖全章函数知识点。

实验要求:

- 任务 1:定义 StatsConfig 配置结构体与 Stats 结果元组(覆盖:无默认参数的替代——配置结构体 §4.7)
- 任务 2:analyze 函数借用输入与配置,一次遍历完成多项统计并返回元组(覆盖:借用参数 &[i32] §4.2、多返回值 §4.3)
- 任务 3:实现带 &self/&mut self 方法的 Counter 计数器类型(覆盖:方法三态 §4.4)
- 任务 4:用闭包实现"偶数计数"(FnMut 捕获可变变量)与"加偏移"(move 捕获)(覆盖:闭包三态 §4.5)
- 任务 5:用 map/filter/fold 组合完成过滤与求和(覆盖:高阶函数 §4.6)
- 任务 6:main 入口读取命令行参数作为数据来源之一,失败给出提示(覆盖:main 与 env::args §4.8、Result 匹配)

实验环境:cargo new stats_pipeline;数据先用固定数组,任务 6 再用 env::args 读取。

第一步 **一步:**先写纯函数 analyze:签名 `fn analyze(data: &[i32], cfg: &StatsConfig) -> Stats`,内部 for + if 过滤

第二步 **二步:**实现 Counter 与它的三个方法;在 main 里用可变方法累加

第三步 **三步:**写 FnMut 闭包(计数)与 move 闭包(偏移),调用并打印

第四步 **四步:**任务 5 用方法链一行完成;与任务 2 的手写循环对比可读性

第五步 **五步:**最后接 env::args:把第 2 个参数解析为 i32 加入数据,失败打印提示

参考答案要点


```
// ===== 实验 4:可配置的数组统计管线 =====
// 覆盖知识点:函数定义与表达式风格($4.1)、借用参数与移动($4.2)、
//             多返回值元组($4.3)、方法三态($4.4)、
//             闭包三态与 move($4.5)、高阶函数 map/filter/fold($4.6)、
//             配置结构体替代默认参数($4.7)、main 与 env::args、Result($4.8)

use std::env;

/// 统计配置:把"默认参数"做成结构体($4.7 的替代方案)
struct StatsConfig {
    title: String,
    min: i32,      // 低于此值忽略
    step: i32,     // 只统计 step 的倍数
}

/// 统计结果:元组即"多返回值"
type Stats = (usize, i32, f64, i32); // (数量, 总和, 平均, 最大)

/// 一次遍历完成所有统计:借用输入与配置,不搬走任何所有权
fn analyze(data: &[i32], cfg: &StatsConfig) -> Stats {
    let mut count: usize = 0;
    let mut sum: i32 = 0;
    let mut max: i32 = i32::MIN;
    for &x in data {
        if x < cfg.min {
            continue;
        }
        if x % cfg.step != 0 {
            continue;
        }
        count += 1;
        sum += x;
        if x > max {
            max = x;
        }
    }
    let avg = if count > 0 { sum as f64 / count as f64 } else { 0.0 };
    (count, sum, avg, max)
}

/// 计数器:方法三态演示(&self 只读、&mut self 修改)
struct Counter {
    total: i32,
    calls: i32,
}

impl Counter {
    /// 关联函数:构造函数惯例
    fn new() -> Self {
        Counter { total: 0, calls: 0 }
    }

    fn add(&mut self, x: i32) {
        self.total += x;
        self.calls += 1;
    }
}
```

```

    }

    fn snapshot(&self) -> (i32, i32) {
        (self.total, self.calls)
    }
}

fn main() {
    // 任务 6 前置:读取命令行参数作为额外数据
    let args: Vec<String> = env::args().collect();
    let extra: Option<i32> = args.get(1).and_then(|s| s.parse().ok());

    let mut data = vec![3, 5, 8, 12, 15, 20, 23, 30, 31, 40, 42];
    match extra {
        Some(v) => data.push(v),
        None => println!("提示:可传入一个整数作为额外数据"),
    }

    // 任务 2:借用配置,返回多值,解构接收
    let cfg = StatsConfig {
        title: String::from("不小于 10 的偶数"),
        min: 10,
        step: 2,
    };
    let (count, sum, avg, max) = analyze(&data, &cfg);
    println!("{count}: 数量 {count}, 总和 {sum}, 平均 {avg:.1}, 最大 {max}", cfg.title);

    // 任务 3:Counter 方法使用
    let mut counter = Counter::new();
    for &x in &data {
        counter.add(x);
    }
    let (total, calls) = counter.snapshot();
    println!("计数器:累计 {total},被调 {calls} 次");

    // 任务 4a:FnMut 闭包—可变捕获计数
    let mut even_count = 0i32;
    let mut tally = |x: i32| {
        if x % 2 == 0 {
            even_count += 1;
        }
    };
    for &x in &data {
        tally(x);
    }
    println!("偶数个数(闭包计数):{even_count}");

    // 任务 4b:move 闭包—把 base 搬进闭包
    let base = 1000;
    let offset = move |x: i32| x + base;
    let shifted: Vec<i32> = data.iter().map(|&x| offset(x)).collect();
    println!("加 {base} 偏移后的前 3 个:{:?}", &shifted[..3]);

    // 任务 5:高阶函数方法链—筛选、变换、归约
    let sum_of_squares: i32 = data.iter()

```

```

        .filter(|&x| x % 3 == 0)      // 只留 3 的倍数
        .map(|&x| x * x)             // 平方
        .fold(0, |acc, x| acc + x);  // 求和
println!("3 的倍数的平方和:{sum_of_squares}");

// 函数指针作参数:与 C 的 qsort 回调对照
let tripled: Vec<i32> = data.iter().map(|&x| triple(x)).collect();
println!("翻三倍的前 3 个:{:?}", &tripled[..3]);
}

/// 普通函数:fn 类型,可传给高阶函数
fn triple(x: i32) -> i32 {
    x * 3
}

```

设计说明:实验把"配置化"贯穿始终——统计规则(阈值、步长)是配置而不是硬编码,这正对应生产里"策略可配置"的需求;闭包与高阶函数负责数据流变换,方法与函数各司其职。验证方式:改变 `cfg.min/step` 观察统计变化;不带参数运行一次,再带参数运行一次,确认 `env::args` 路径生效。

下一章预告:第5章 内存管理——全册的灵魂章节。C 的 `malloc/free` 时代有三大痛点:泄漏、悬垂指针、重复释放;Rust 用"所有权三法则 + 借用 + 生命周期"把这三类事故全部变成编译错误。`Box/Rc/Arc/RefCell` 各司其职,`unsafe` 划定安全边界。这一章读透了,后面的每一章都会轻松很多。